GoNature — Complete Project Breakdown

File-by-file reference for demo & defense · Group 9

> **The big picture:** Four projects. `G9_Shared` holds the data objects (DTOs) that travel over the wire. `OCSF` is the networking framework (sockets). `G9_Prototype_Server` is the brain — it talks to MySQL and enforces all rules. `G9_Prototype_Client` is the JavaFX GUI. Client and server only ever exchange `Message` objects; the client **pulls** data by asking and waiting for a reply (no live push).

1. Shared / DTO Layer `G9_Shared/src/logic`

Plain `Serializable` value objects — the "nouns" sent between client and server. Bundled into **both** jars so both sides agree on the format.

Protocol envelope

- **Message.java** — The single envelope for *all* communication. Holds a `type` string (the command, e.g. `"BOOK_ORDER"`) and an `Object data` payload. Every packet over the socket is a `Message`. `getType()` is the discriminator the big switch statements read.

Requests (client → server)

- **BookingRequest.java** — A new-booking ask: visitor ID, park, date/time, party size, type, email. Payload of `BOOK_ORDER` and `JOIN_WAITING_LIST`.
- **EntryCheckRequest.java** — Gate check-in ask. If a booking exists the server ignores the count/type fields and uses the booking's. Payload of `CHECK_IN_VISITOR`.
- **ExitRequest.java** — "This visitor is leaving" (visitor ID + park). Payload of `REGISTER_EXIT`.
- **RegisterRequest.java** — New visitor sign-up (ID, name, email, phone, optional subscriber + family size).
- **RegisterGuideRequest.java** — Service rep registering a tour guide (inserted as pending approval).
- **ReportRequest.java** — Report filter: park (0 = all), year, month. Used by all the report queries.
- **ParkSettingsRequest.java** — Park manager's proposed capacity/price/duration change. Dual-purpose: submission payload **and** a row in the dept-manager's approval table. `getSummary()` formats it for display.
- **Order.java** — Lightweight edit payload (order #, new date, new count, type) for `UPDATE_ORDER`. Distinct from the read-only `OrderDetail`.

Results (server → client)

- **LoginResult.java** — Who logged in: role, name, email, ID, optional park. Stored in `UserSession`.
- **BookingResult.java** — Booking confirmation: status (`CONFIRMED` / `WAITING`), details, **plus the price preview** (base, final, promo % and text). `hasPromotion()` tells the UI whether to show the discount line.
- **EntryResult.java** — Check-in outcome: approved?, pre-booked vs walk-in, price, message. Shown on the entry card.
- **CancelResult.java** — Cancel outcome + `getNotifiedEmail()` (the waiting-list person who got promoted into the freed slot, or null).
- **RegisterResult.java** — Sign-up outcome + generated subscriber ID if applicable.

Read-only projections (display rows)

- **Park.java** — A park's config (capacity, max orders, visit duration, full price). `toString()` returns the name so it renders directly in combo-boxes.
- **OrderDetail.java** — The full read-only order row used everywhere orders are listed (My Orders, Today's Reservations, etc.). Has an extended constructor carrying a `confirmationDeadline` for waitlist-promotion orders.
- **GuideDetail.java** — A guide row for the Manage Guides panel. `getFullName()` / `getStatusLabel()` format for display; has a self-contained `capitalize()` so it needs no client-only code.
- **Promotion.java** — A discount promotion row. Constants `PENDING` / `ACTIVE` / `REJECTED`. Two constructors (full DB read vs. fresh submission). `getDiscountLabel()` / `getDateRange()` format for display.

Report rows

- **ReportVisitorRow.java** — One day's visitor total, split SOLO/GROUP → visitor bar chart.
- **ReportCancelRow.java** — One cancelled/no-show order → cancellation table.
- **ReportCancelDistribution.java** — Cancellations bucketed across the 7 weekdays. Computes `getAvgPerWeekday()`, `getPeakDay()`, `getPeakCount()` for the day-of-week chart.
- **ReportUsageRow.java** — Average concurrent load at one hour slot vs. capacity → usage line chart.
- **ReportDurationRow.java** — Average visit length (hours) per visitor type → duration bar chart.

2. Communication Layer OCSF/src

Object Client-Server Framework — a textbook library (don't edit). Sends Java objects over TCP sockets. **Not included in the Javadoc** because it's third-party.

- **AbstractServer.java** — Base server. Runs a listener thread that accepts clients. Key: `listen()`, `stopListening()`, `close()`, `sendToAllClients()` (exists but unused — we're pull-based), and the **abstract** `handleMessageFromClient()` + override hooks `serverStarted()`, `clientConnected()`, `clientDisconnected()`, `clientException()`. `EchoServer` extends this.
- **ConnectionToClient.java** — One per connected client (its own thread). `sendToClient(obj)` writes back to that client. `setInfo()/getInfo()` stash per-connection data — we use it to remember each client's session key and IP.
- **AbstractClient.java** — Base client. `openConnection()`, `sendToServer(obj)`, `closeConnection()`, and the **abstract** `handleMessageFromServer()` that runs on a background reader thread. `ChatClient` extends this.
- **Supporting framework files** (unused by us but part of the library): `AdaptableServer`, `AdaptableClient`, `ObservableServer`, `ObservableClient`, `ObservableOriginatorServer`, `OriginatorMessage`, plus `module-info.java`.

3. Server Layer G9_Prototype_Server/src

Entry & lifecycle

- **ServerLauncher.java** — Jar `Main-Class`. A plain (non-JavaFX) class that just calls `ServerUI.main()`. This trick sidesteps the "JavaFX runtime components are missing" launcher guard.

- **ServerUI.java** — JavaFX app entry. Shows the **DB password dialog** (password is typed at runtime, never stored on disk), tests the connection, then opens the control panel. `runServer(port)` spins up `EchoServer` on a daemon thread.
- **ServerPortFrameController.java** — Controller for the server control panel (port field, connected-clients table, connection log, simulation log). All its `static` helpers (`addClient`, `logSimulation`, `upgradeReminderToSent`, etc.) are thread-safe via `Platform.runLater`. `Done()` starts the server; `stopServer()` stops it; `handleExportLogs()` saves logs to a file.

The router

- **EchoServer.java** — **The heart of the server.** `handleMessageFromClient()` is one giant `switch` on message type (~40 commands) that calls the right `DBController` method and sends a typed reply. Highlights:
  - **Single-login enforcement:** `activeSessions` set rejects the same account logging in twice.
  - **Overbooking protection:** `BOOK_ORDER` grabs a **per-park lock** (`parkLocks`) before calling `checkAndBook()` — two layers of safety with the DB's `SELECT … FOR UPDATE`.
  - **Server-side validation:** `validateBookingRequest()` re-checks every rule (SOLO=1, FAMILY/GROUP=2–15, guides up to 16) even though the client also validates.
  - `serverStarted()` opens the DB and arms the three background services (Reminder, Waitlist, No-show).

Background services (threads — the "runs forever without restart" story)

- **ReminderService.java** — Day-before reminders. `runStartupCheck()` runs once immediately, then `scheduleAtFixedRate` re-runs **every day at 00:05** so the server never needs restarting. `secondsUntilNextDailyRun()` aligns it to midnight; the try/catch in `runReminderCheck()` keeps a transient DB error from killing the daily schedule. Also handles the 2-hour auto-cancel of unconfirmed orders.
- **WaitlistTimerService.java** — Enforces the **1-hour confirmation window** for promoted waiting-list visitors. Two mechanisms: a precise one-shot timer per promotion **and** a 5-minute safety sweep (survives restarts). Both funnel through the idempotent `expireWaitlistEntry()`.
- **NoShowService.java** — Every 5 min, finds confirmed orders whose visit time + 15-min grace passed with no check-in, marks them `no_show`, and promotes the waiting list.
- **EmailService.java** — Sends real Gmail SMTP emails (SSL port 465). Credentials read once from `db.properties`. Each send runs on a bounded daemon thread-pool so SMTP never blocks a request; failures are logged and swallowed.
- **ReminderOrderInfo.java** — Tiny server-only struct (not serialized) describing one order needing a reminder.

Database

- **DBController.java** — **The data-access layer** (singleton, ~3000 lines, the largest file). Every SQL query lives here as a parameterized `PreparedStatement`. Grouped by feature:
  - *Connection:* `connect()`, `isConnected()`, `setPassword()`.
  - *Parks:* `getParks()`, `getParkById()`, `getAvailableSpots()`, `getCurrentOccupancy()`.
  - *Booking (concurrency-critical):* `checkAndBook()` (uses `SELECT … FOR UPDATE`), `addToWaitingList()`, `findAlternativeSlots()`, `quotePrebookedPrice()`.
  - *Orders:* `getOrdersByVisitor()`, `getOrderById()`, `cancelOrder()`, `updateOrder()`, `getTodayBooking()`, `getAllLiveOrders()`, `getTodayOrders()`.
  - *Check-in/exit:* `checkInVisitor()`, `registerExit()`.
  - *Waiting list:* `promoteFromWaitingList()`, `markNoShowsAndPromote()`, `expireWaitlistEntry()`, `confirmPromotedVisit()`.
  - *Auth:* `loginVisitor()`, `loginUser()`, `isRegisteredGuide()`, `isApprovedGuide()`.
  - *Reports:* `getVisitorReport()`, `getCancelReport()`, `getCancellationDistribution()`, `getUsageReport()` (the interval-overlap occupancy query), `getVisitDurationByType()`.
  - *Approvals:* settings (submit/approve/reject), promotions (submit/approve/reject, `getBestActivePromotion()`), guides (register/approve/reject).

4. Client Layer G9_Prototype_Client/src

Network plumbing (`client` package)

- **ClientLauncher.java** — Jar `Main-Class`; same launcher trick → calls `ClientUI.main()`.
- **ClientUI.java** — JavaFX entry. Prompts for the **server IP**, creates the shared `ClientController`, then shows the login screen. The static `chat` field is the one connection every screen uses.
- **ClientController.java** — Thin façade over `ChatClient`. `accept(msg)` sends-and-blocks; `disconnect()` / `sendLogout()` for teardown.
- **ChatClient.java** — **The client's core.** Extends OCSF `AbstractClient`. Key pattern: a **blocking request/response** built on shared static fields. `handleMessageFromClientUI()` sets `awaitResponse=true`, sends, and **spins until the reply arrives** (15-sec safety timeout). `handleMessageFromServer()` (on the reader thread) routes each reply type into the matching `lastXxx` static field, then clears `awaitResponse`. The GUI then reads that field.
- **ChatIF.java** — One-method interface (`display`) decoupling the network layer from any UI widget.
- **UserSession.java** — Singleton holding the logged-in user (role, name, park…). Screens read identity from here instead of passing it around. `getSessionDuration()` powers the on-screen clock; `clear()` on logout.
- **NotificationCenter.java** — In-memory notification inbox (simulated SMS/email), capped at 50, newest-first. Also tracks which orders the visitor already confirmed so the Confirm button stays disabled.

Screen controllers (`gui` package) — each pairs with an `.fxml`

- **LoginController.java** — Single login form; auto-detects visitor/guide (ID only) vs. staff (username+password) from the server's role reply.
- **RegisterController.java** — New visitor sign-up (subscriber sign-up deliberately lives in the service-rep screen per spec).

- **VisitorController.java** — Visitor/guide home: welcome, action buttons, notification inbox, auto-reminder for tomorrow's visits.
- **BookingController.java** — The booking form: park/slot loading, live price estimate, client-side validation (mirrors server), waiting-list offer when full.
- **BookingConfirmationController.java** — Confirmation screen; `populate(BookingResult)` fills it. Shows the entry-pass card (with QR) for confirmed bookings, hides it for waitlist.
- **MyOrdersController.java** — Visitor's orders with colour-coded status; cancel enabled only for PENDING/CONFIRMED; popup if a waitlist visitor got promoted.
- **EmployeeController.java** — Park employee/manager home; gates access to Today's Reservations, Entry Control, Waiting List, and (managers only) Reports/Settings/Promotions.
- **EntryControlController.java** — Gate screen: Check-In (pre-booked or walk-in) and Register Exit tabs.
- **TodayReservationsController.java** — Today's bookings for the employee's park (`GET_TODAY_ORDERS`).
- **WaitingListController.java** — Queued visitors for the park (`GET_WAITING_LIST`).
- **ParkSettingsController.java** — Park manager edits capacity/price/duration/max-orders → submits for dept-manager approval.
- **PromotionsController.java** — Park manager proposes discounts (stored PENDING until approved).
- **ServiceRepController.java** — Service-rep desk: load/update orders for a visitor, register new visitors/subscribers.
- **AcademicFrameController.java** — Department-manager screen: loads all live orders (`GET_LIVE_ORDERS`), edits any order by ID.
- **ReportsController.java** — Reports dashboard: visitor/cancellation/usage/duration tabs with a shared month-year selector. Managers locked to their park; dept managers see all.

UI helpers (`gui` package) — no FXML, pure styling/utility

- **ThemeManager.java** — App-wide light/dark toggle. Watches `Window.getWindows()` so every current/future screen follows the theme. `installToggle(Button)` wires any button as the switch.
- **UiPolish.java** — Idempotent scene decorator: swaps emoji for the vector pine logo and adds button hover/press motion to every screen automatically.
- **Icons.java** — Vector (SVGPath) icon factory replacing washed-out Windows emoji; colour comes from CSS classes.
- **Animations.java** — Small fade/slide transition helpers (JavaFX has no CSS transitions).
- **Toast.java** — Floating top-right "toast" pills for one-shot outcomes; stack and auto-dismiss (~3.2s).
- **HelpDialog.java** — Shared themed Help popup (`show(title, body)`) used by every screen's "Help ?" button.
- **EmptyState.java** — Builds the "no data" placeholders (icon + title + hint) for empty tables.
- **UiCells.java** — `applyStatusChip()` renders order-status text as colour-coded pill chips in tables.
- **QRCodeUtil.java** — **Simulated** QR generator for entry passes. Encodes `GONATURE:ORDER=<id>:VISITOR=<id>` , which the scan dialog can parse to auto-fill the visitor ID. (Not a real QR code.)

FXML layout files (`gui` package)

15 `.fxml` files define each screen's visual layout, each paired with its controller above: `LoginFrame` , `RegisterFrame` , `VisitorFrame` , `BookingFrame` , `BookingConfirmationFrame` , `MyOrdersFrame` , `EmployeeFrame` ,

`EntryControlFrame`, `TodayReservationsFrame`, `WaitingListFrame`, `ParkSettingsFrame`, `PromotionsFrame`, `ServiceRepFrame`, `AcademicFrame`, `ReportsFrame`. (Styling lives in `gonature.css`.)

One-paragraph flow for your defense

A user enters the **server IP** → `ClientUI` opens one socket via `ChatClient`. They log in: `LoginController` sends `LOGIN_VISITOR` / `LOGIN_USER`, `EchoServer` calls `DBController.loginVisitor()`, replies `LOGIN_SUCCESS` with a `LoginResult` stored in `UserSession`. Booking: `BookingController` sends a `BookingRequest`; `EchoServer` takes the **per-park lock**, calls `checkAndBook()` (with `SELECT … FOR UPDATE`), and either confirms (with a price preview + confirmation email) or offers the waiting list. Meanwhile three **daemon threads** run on the server forever — `ReminderService` (daily 00:05), `WaitlistTimerService` (1-hour windows), `NoShowService` (5-min sweeps) — so the system behaves correctly for weeks without a restart. Every message in and out is a `Message`, and the client always **asks and waits** rather than being pushed to.